Table of Contents

1 Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

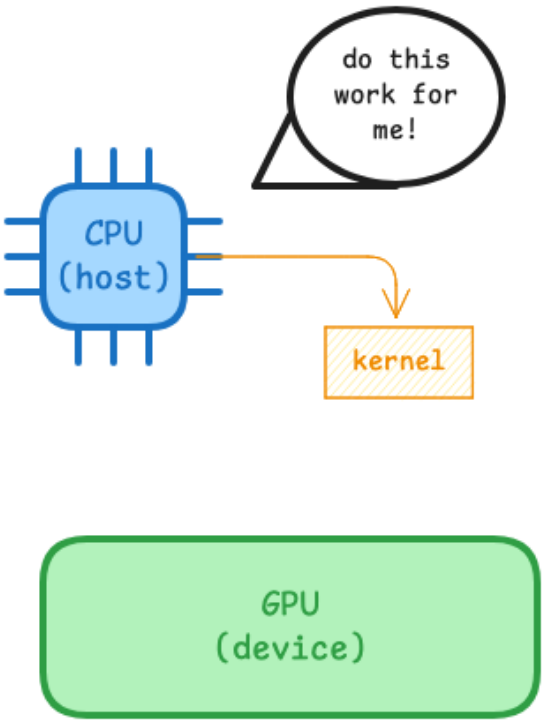
```
sgemm(
  int M, int N, int K,
  float alpha, float *A, float *B,
  float beta, float *C
)
```

$$\underbrace{A}_{M \times K} \quad \underbrace{B}_{K \times N} \quad \underbrace{C}_{M \times N}$$

- M: rows of A and C
- N: columns of B and C
- K: shared reduction dimension

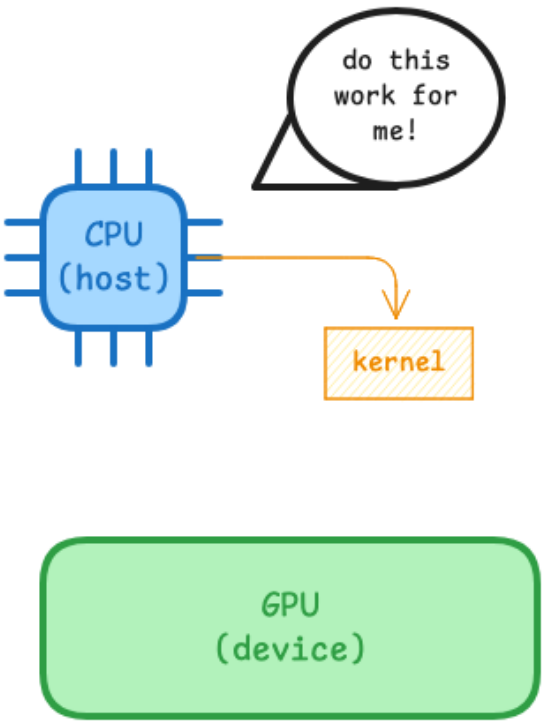
CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.



CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

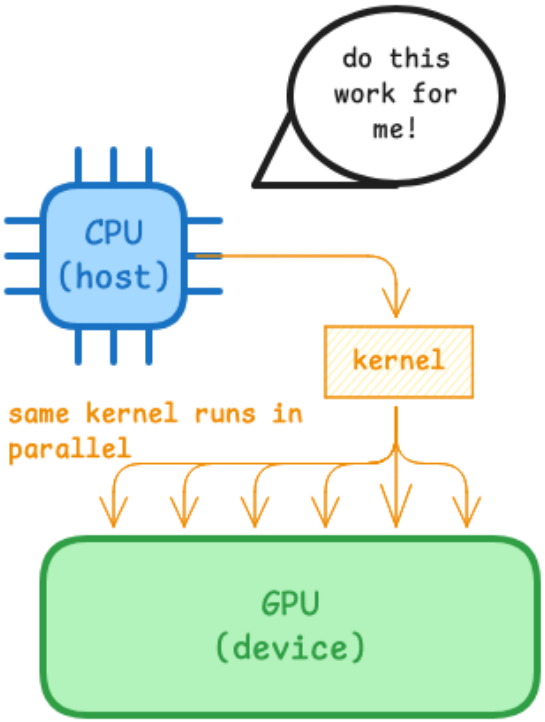
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

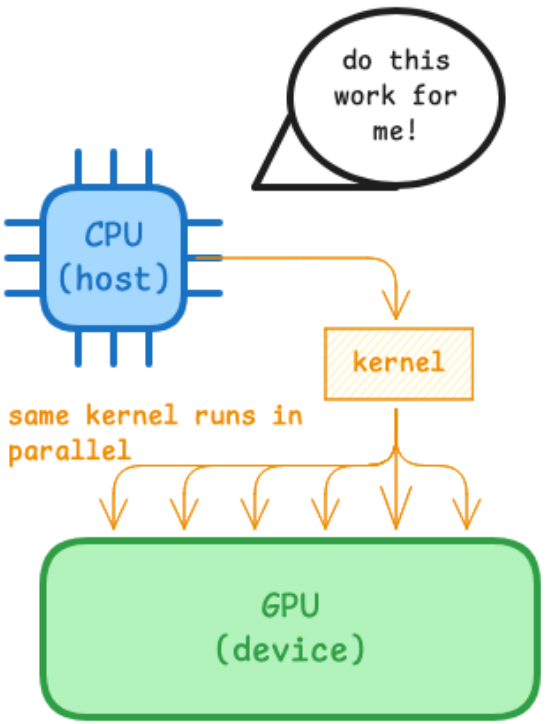
CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.

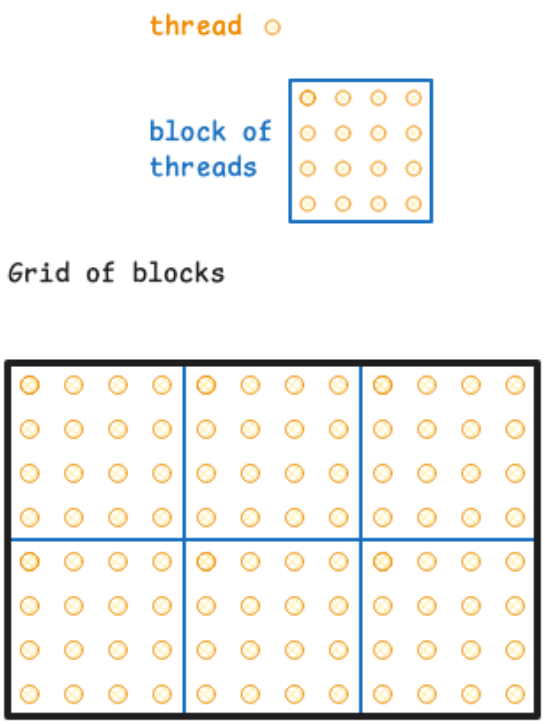



```
__global__ void sgemm_naive(
  int M, int N, int K, float alpha, const float *A,
  const float *B, float beta, float *C
) {
  // implement kernel ...
}

void run_sgemm_naive(int M, int N, int K, float alpha,
     float *A, float *B,
                        float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

CUDA Basics (Threads, Blocks, Grids)

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.




```
__global__ void sgemm_naive(
  int M, int N, int K, float alpha, const float *A,
  const float *B, float beta, float *C
) {
  // implement kernel ...
}
```

```
void run_sgemm_naive(int M, int N, int K, float alpha,
     float *A, float *B,
                       float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Why Threads, Blocks, Grids?

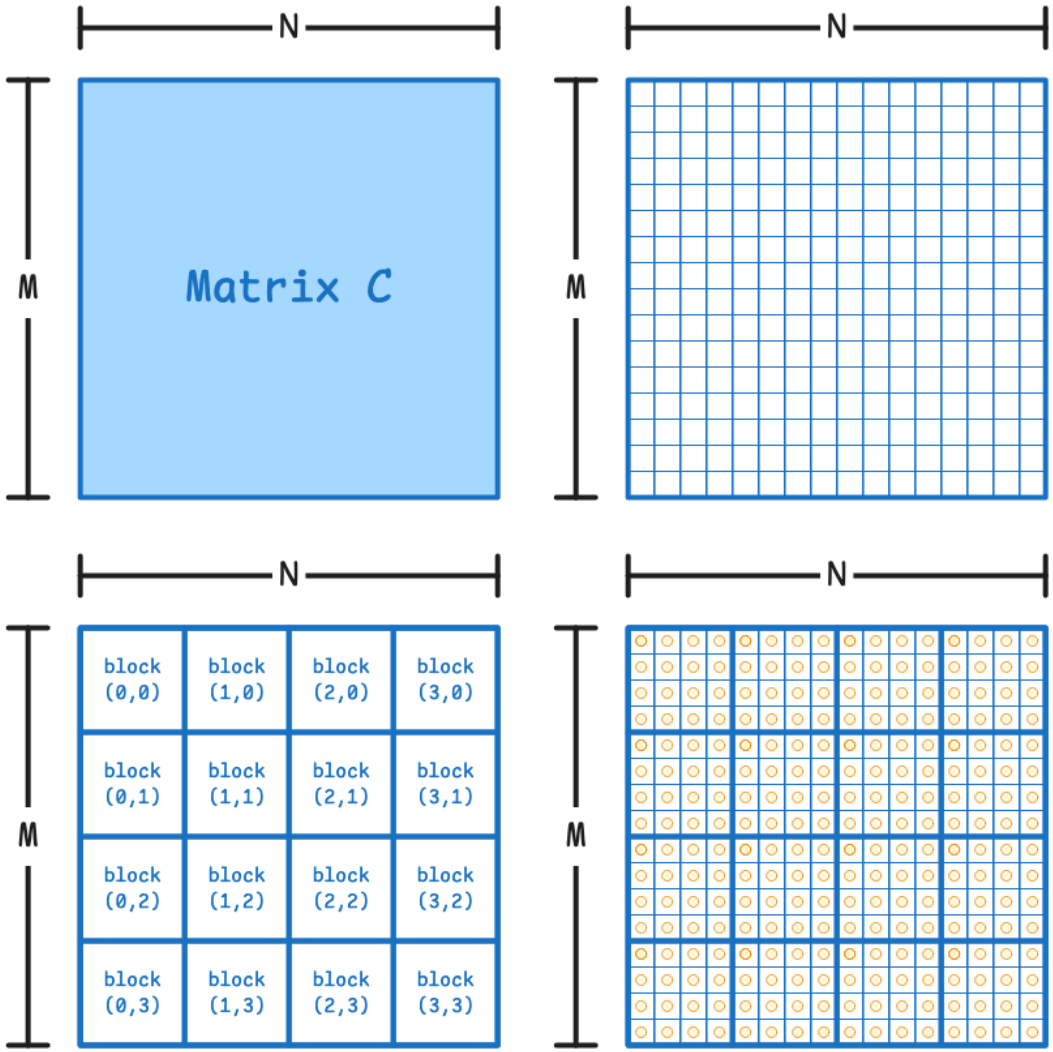
This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution. We will see later!

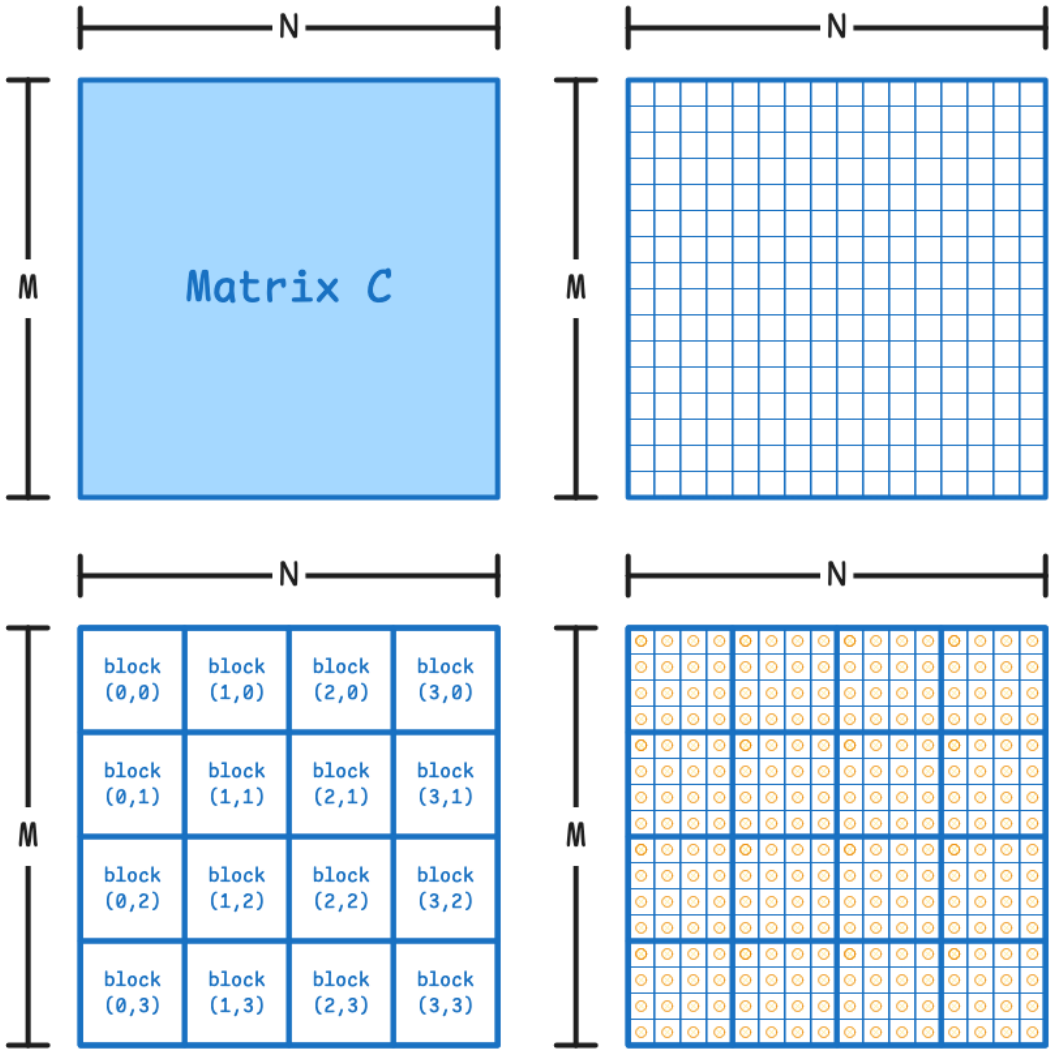
Naive SGEMM: map the output to CUDA

1. Divide C into tiles—one tile per CUDA block.



Naive SGEMM: map the output to CUDA

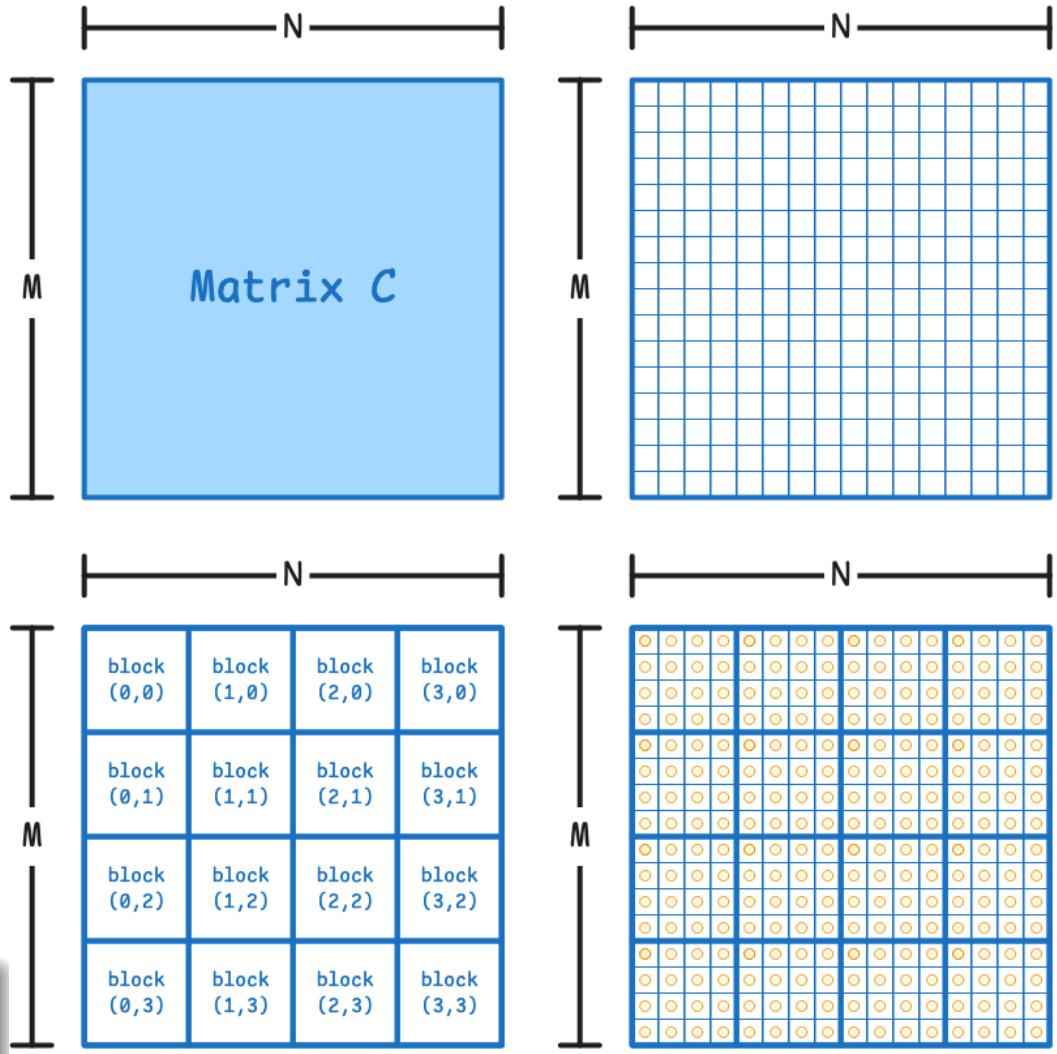
1. Divide C into tiles—one tile per CUDA block.
2. Put one thread on every element in the tile.



Naive SGEMM: map the output to CUDA

1. Divide C into tiles—one tile per CUDA block.
2. Put one thread on every element in the tile.
3. Each thread independently computes one dot product:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$



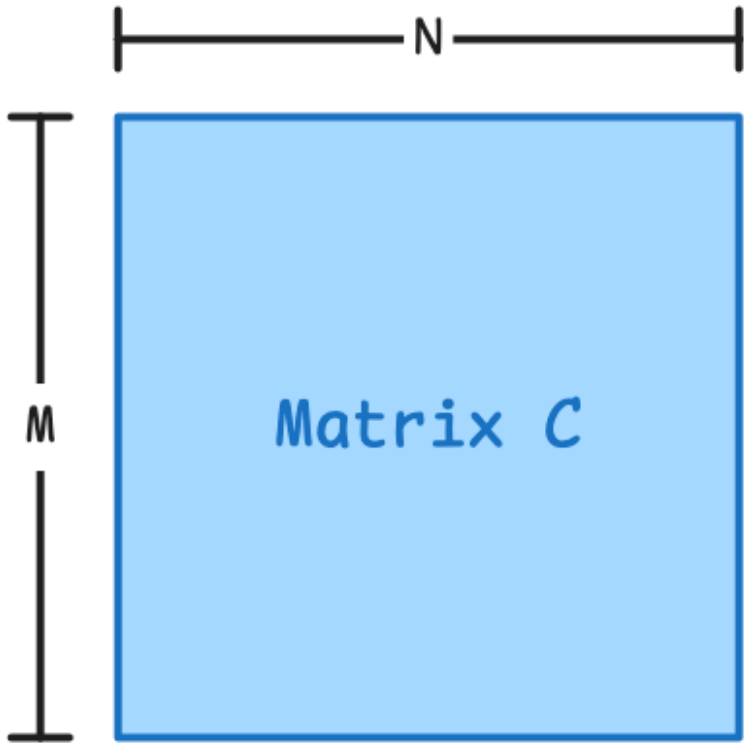
Simple parallelism

$M \times N$ outputs become $M \times N$ independent threads.

Naive SGEMM Kernel

```
__global__ void sgemm_naive(
    int M, int N, int K, float alpha, const float *A,
    const float *B, float beta, float *C
) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

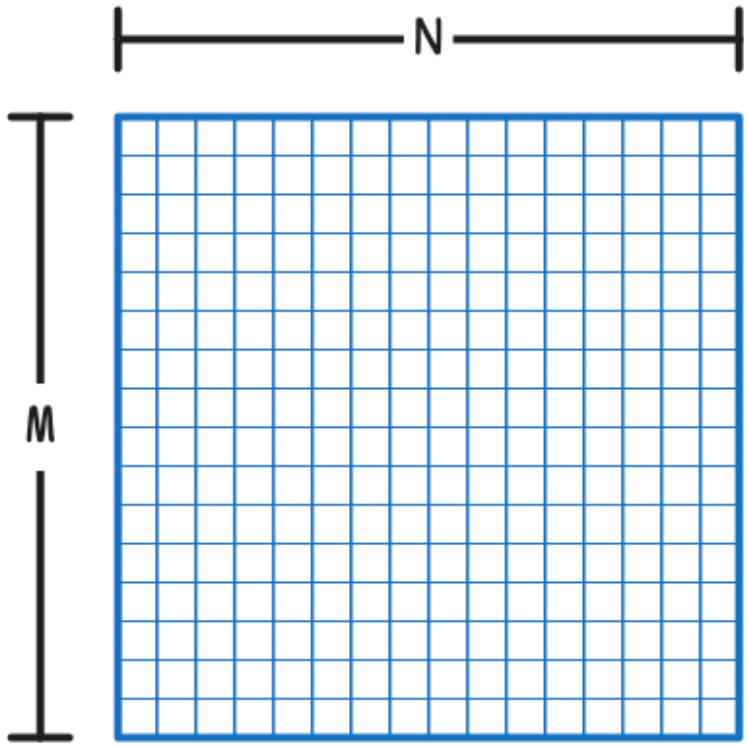
  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}
void run_sgemm_naive(
  int M, int N, int K, float alpha, float *A,
  float *B, float beta, float *C
) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```



Naive SGEMM Kernel

```
__global__ void sgemm_naive(
    int M, int N, int K, float alpha, const float *A,
    const float *B, float beta, float *C
) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

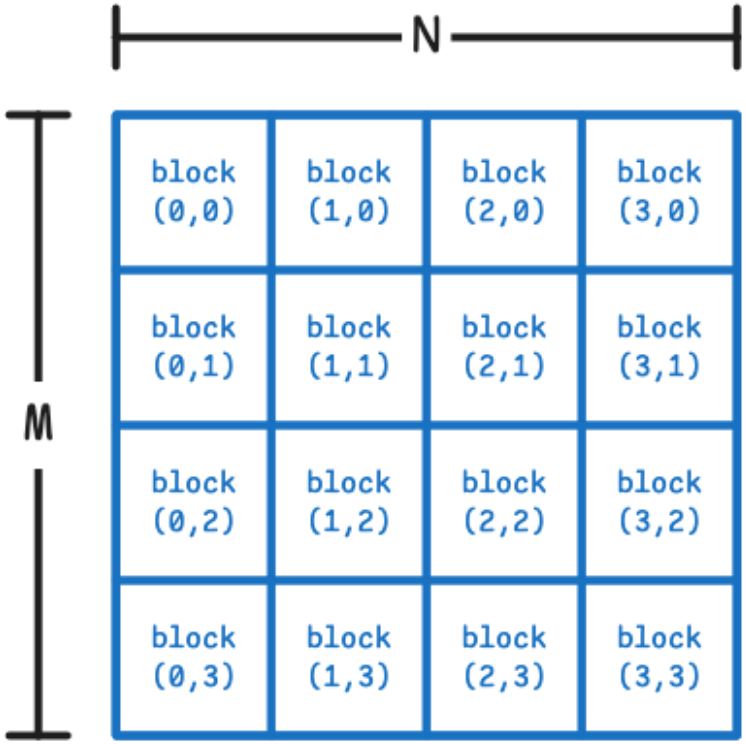
  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}
void run_sgemm_naive(
  int M, int N, int K, float alpha, float *A,
  float *B, float beta, float *C
) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```



Naive SGEMM Kernel

```
__global__ void sgemm_naive(
    int M, int N, int K, float alpha, const float *A,
    const float *B, float beta, float *C
) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

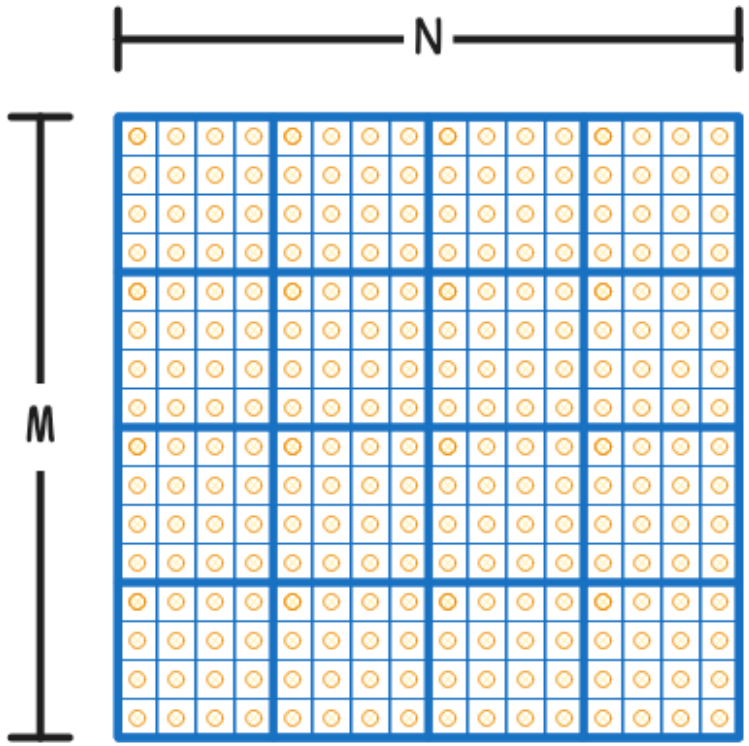
  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}
void run_sgemm_naive(
  int M, int N, int K, float alpha, float *A,
  float *B, float beta, float *C
) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```



Naive SGEMM Kernel

```
__global__ void sgemm_naive(
    int M, int N, int K, float alpha, const float *A,
    const float *B, float beta, float *C
) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}
void run_sgemm_naive(
  int M, int N, int K, float alpha, float *A,
  float *B, float beta, float *C
) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

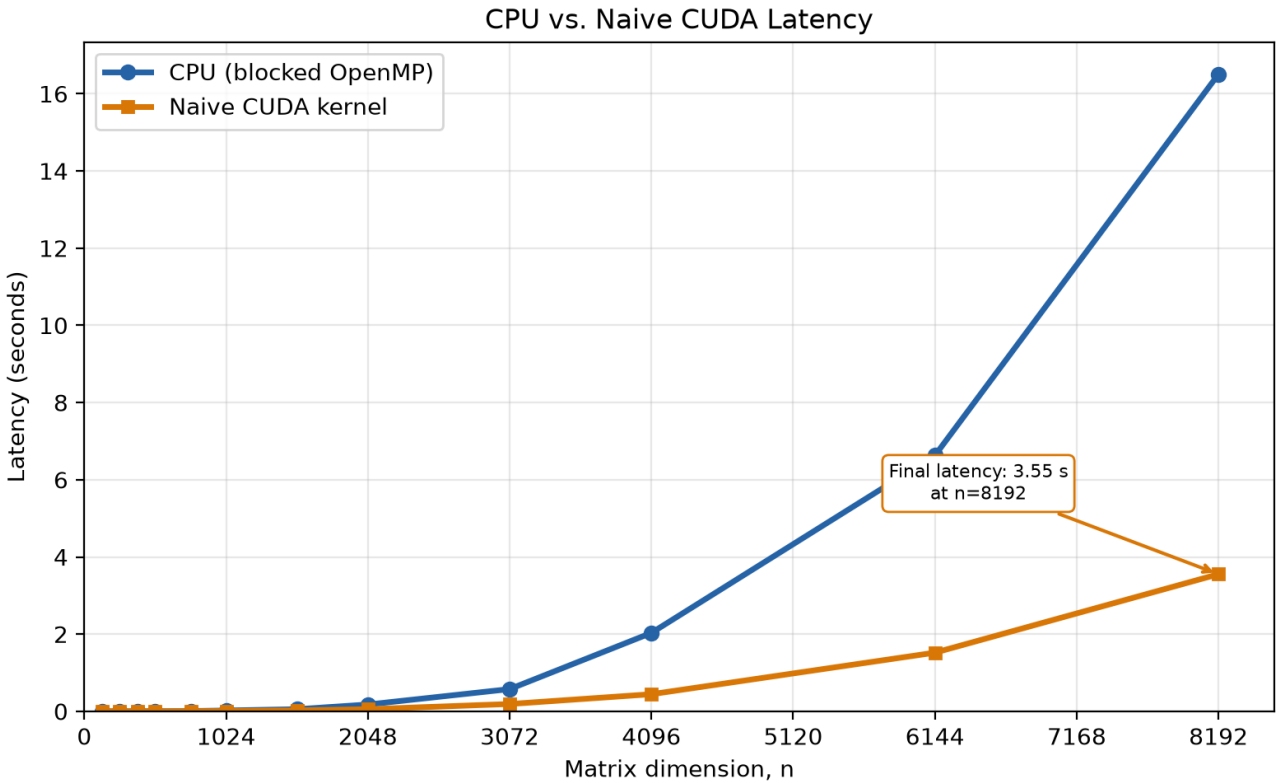


Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.

Play naive-kernel visualization

Naive CUDA is already faster than the CPU



We can do better

Observed latency: 3.55s.

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.
Therefore, if we achieve peak FP32 throughput, we should have a latency of

$$\frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031\text{s} = 31\text{ms}.$$

We can do better

Observed latency: 3.55s.
A SGEMM with $m, n, k = 8192$ requires $2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.
Therefore, if we achieve peak FP32 throughput, we should have a latency of

$$\frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031\text{s} = 31\text{ms}.$$

Whats the hold up?

Our observed latency of 3.55 *seconds* is more than 100× the theoretical minimum of 31 *milliseconds*!

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, so we have $m \times n \times (2k) = 2(8192)^3$ DRAM accesses for the matmul portion. At FP32, that is $2(8192)^3 \times 4$ bytes ≈ 3298.5 GB.

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, so we have $m \times n \times (2k) = 2(8192)^3$ DRAM accesses for the matmul portion. At FP32, that is $2(8192)^3 \times 4$ bytes ≈ 3298.5 GB.
The RTX 3090 has a DRAM bandwidth of 935.8 GB/s. So the time required to serve all the DRAM requests is $3298.5/935.8 \approx 3.52$s.

We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, so we have $m \times n \times (2k) = 2(8192)^3$ DRAM accesses for the matmul portion. At FP32, that is $2(8192)^3 \times 4$ bytes ≈ 3298.5 GB.
The RTX 3090 has a DRAM bandwidth of 935.8 GB/s. So the time required to serve all the DRAM requests is $3298.5/935.8 \approx 3.52$s.

We are memory bound!

Our observed latency of 3.55 seconds is dominated by the time it takes to move the data from DRAM to the CUDA Cores, so we are **memory bound**. Our CUDA Cores are sitting idle for most of the time!

Shared Memory (SMEM) Caching

Programmable L1 Cache (SMEM)

Remember how our launch config. required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so threads in the same block have access to the SM's L1 cache/SMEM.

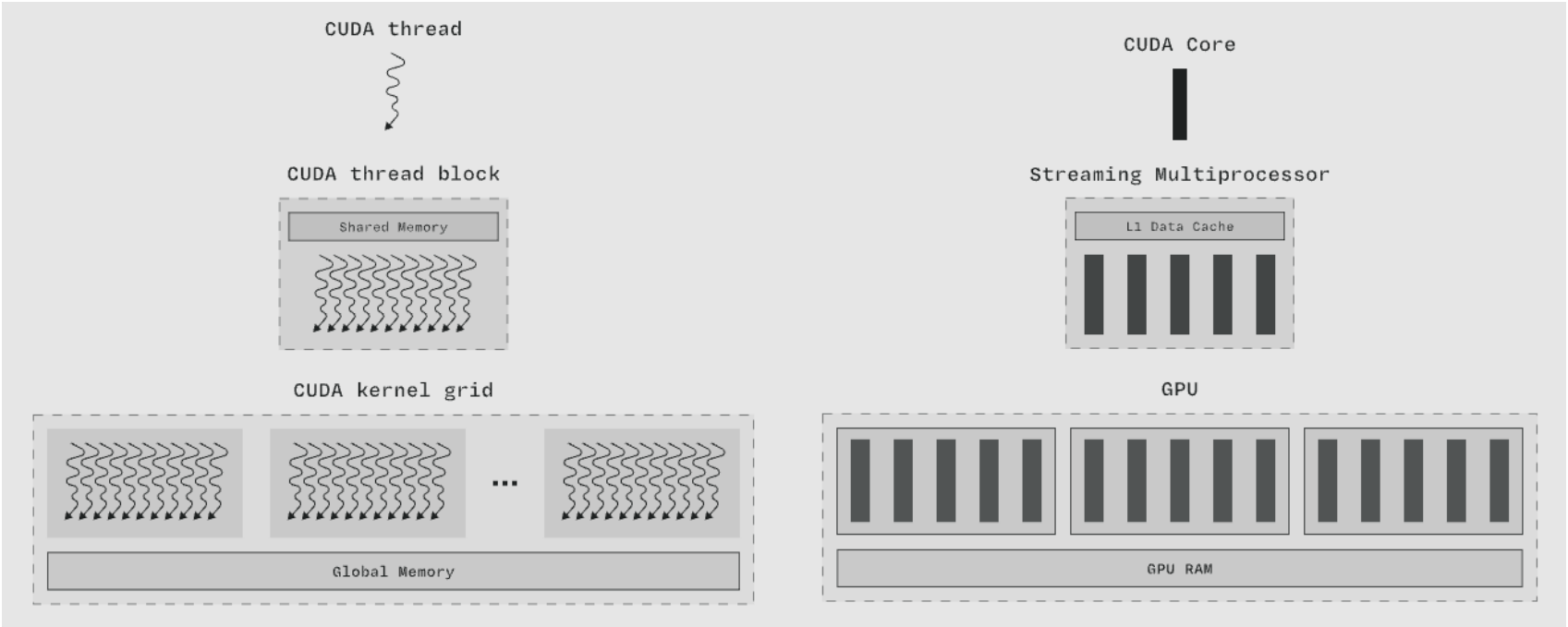


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

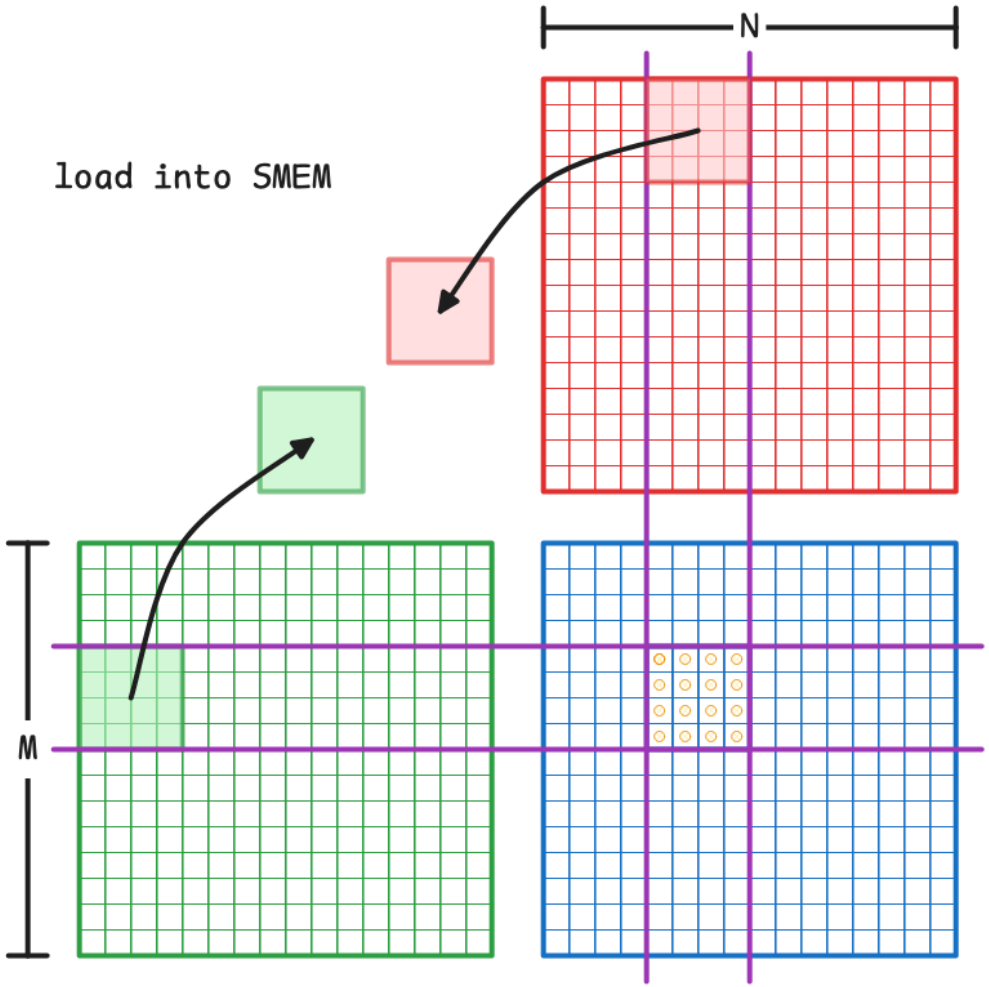
Tiling

Threads can save time by loading data from Device/Global memory into Shared Memory within the SM.

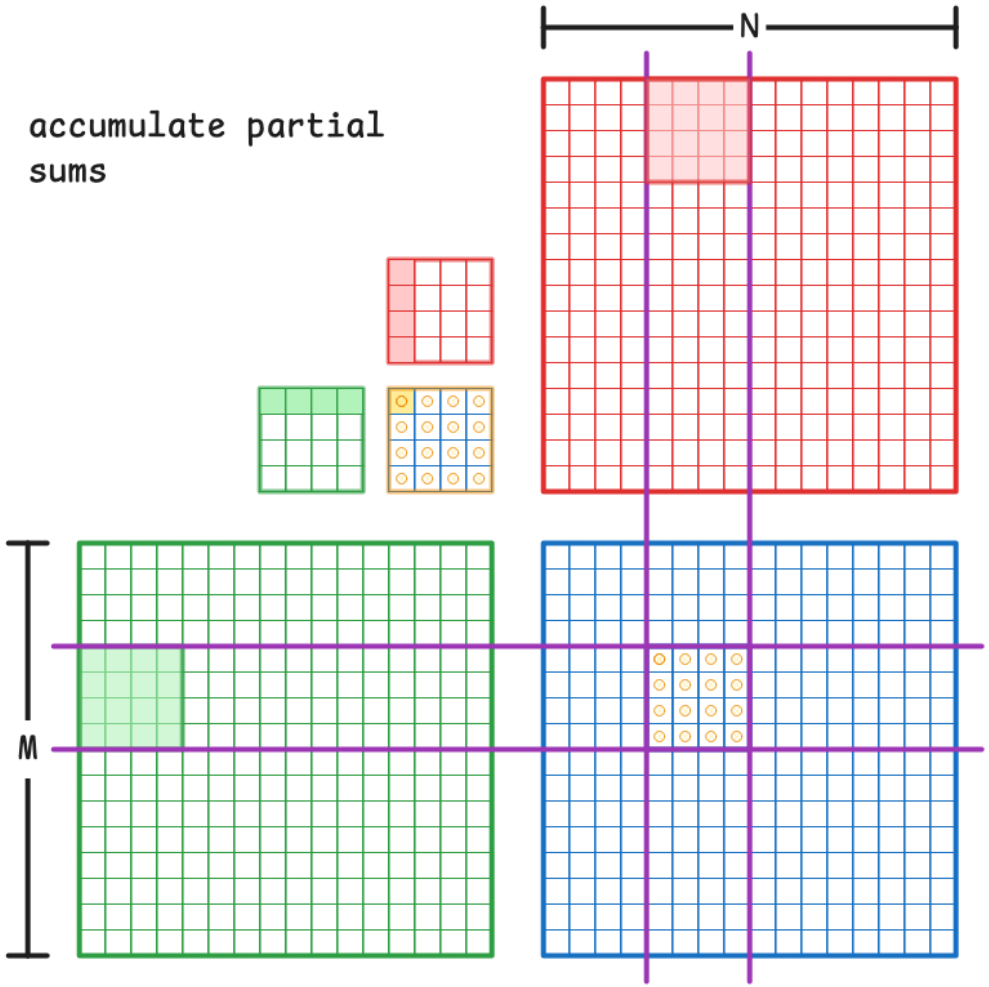
Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

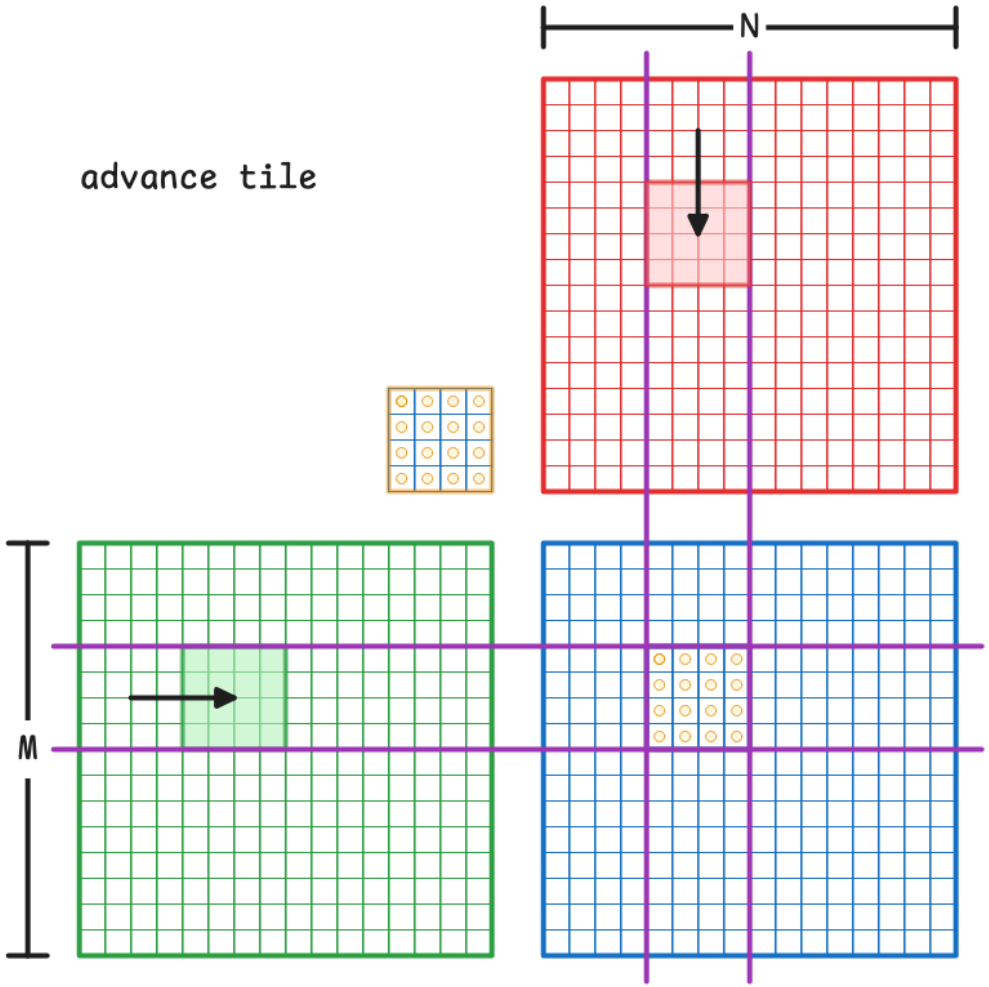
Tiling steps



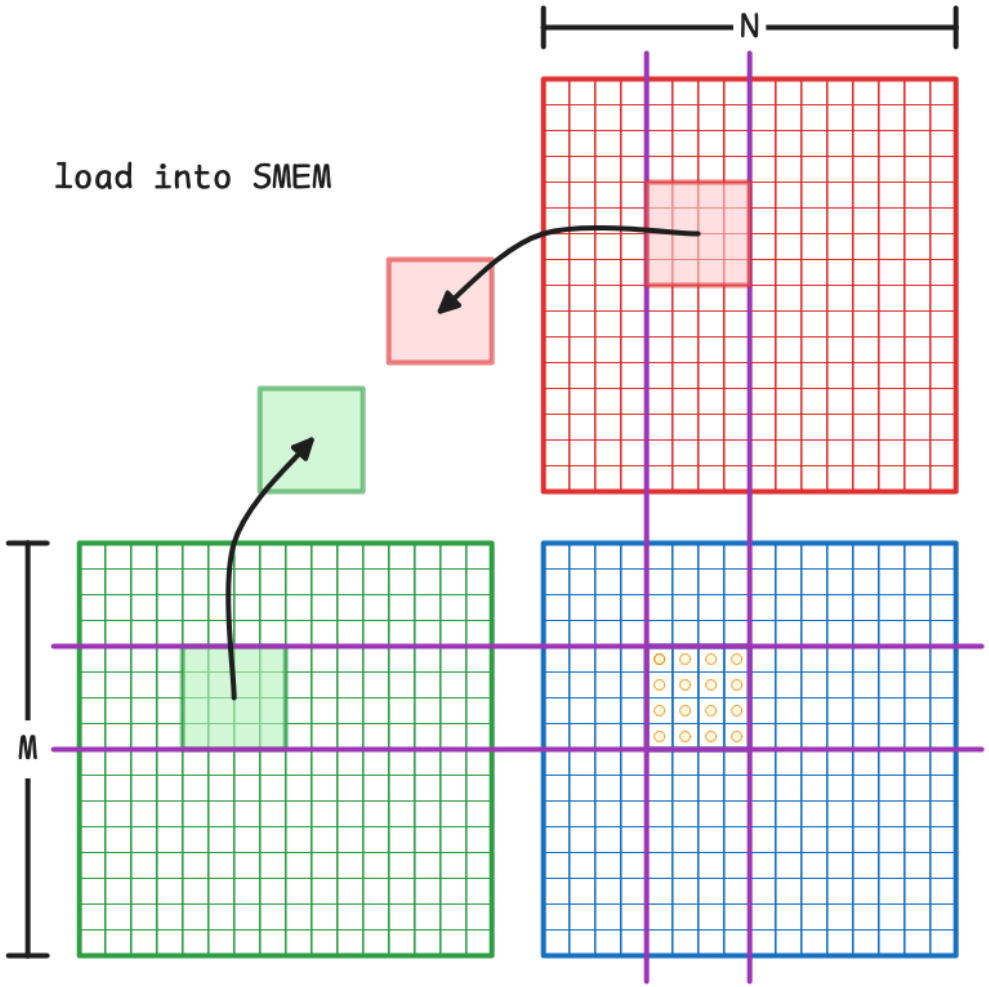
Tiling steps



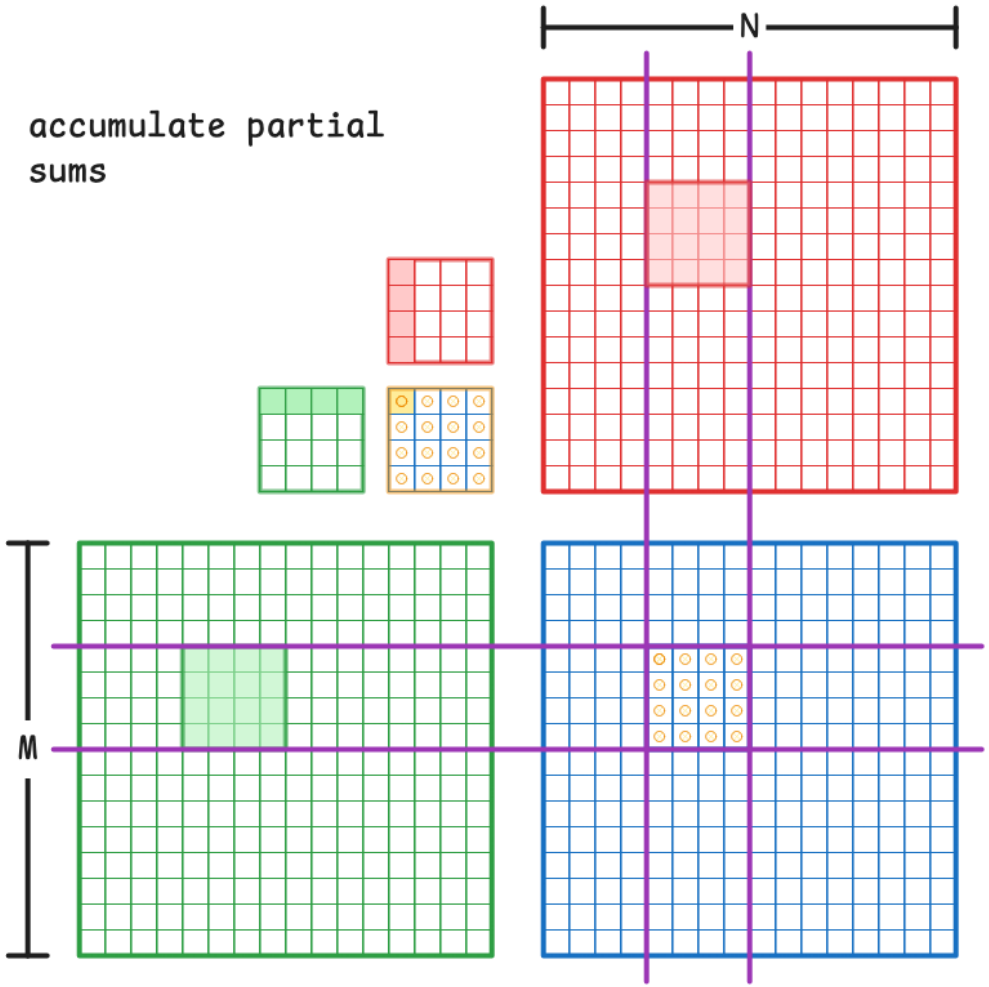
Tiling steps



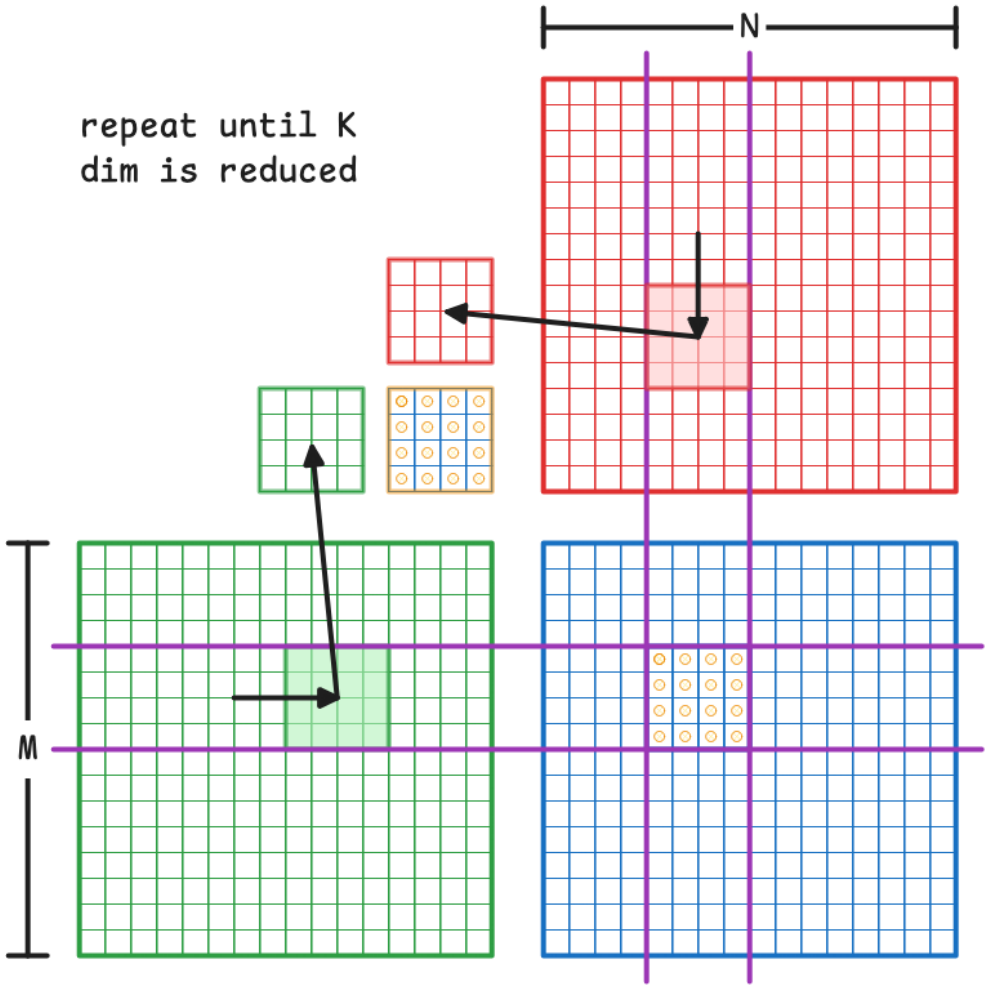
Tiling steps



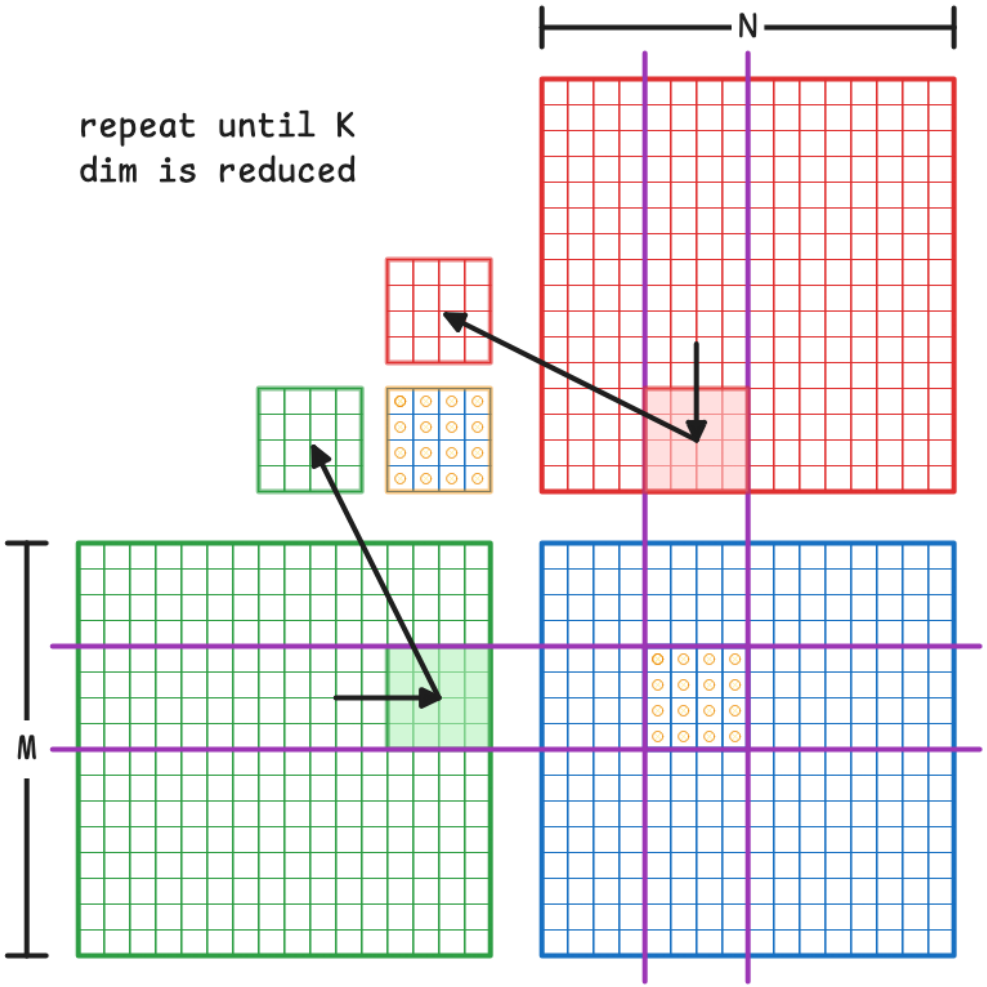
Tiling steps



Tiling steps



Tiling steps



Basic tiled SGEMM

```
__shared__ float As[BLOCKSIZE * BLOCKSIZE];
__shared__ float Bs[BLOCKSIZE * BLOCKSIZE];
float tmp = 0.0;

for (int bkIdx = 0; bkIdx < K; bkIdx += BLOCKSIZE) {
  As[threadRow * BLOCKSIZE + threadCol] =
      A[threadRow * K + threadCol];
  Bs[threadRow * BLOCKSIZE + threadCol] =
      B[threadRow * N + threadCol];

  __syncthreads();
  A += BLOCKSIZE;
  B += BLOCKSIZE * N;

  for (int dotIdx = 0; dotIdx < BLOCKSIZE; ++dotIdx) {
    tmp += As[threadRow * BLOCKSIZE + dotIdx] *
           Bs[dotIdx * BLOCKSIZE + threadCol];
  }

  __syncthreads();
}
```

Cooperate
Each thread loads one value from A and one from B.

Synchronize
Wait until both shared tiles are complete.

Reuse
One thread accumulates from the cached row and column.

Advance
Protect the cache, move along K, and repeat.

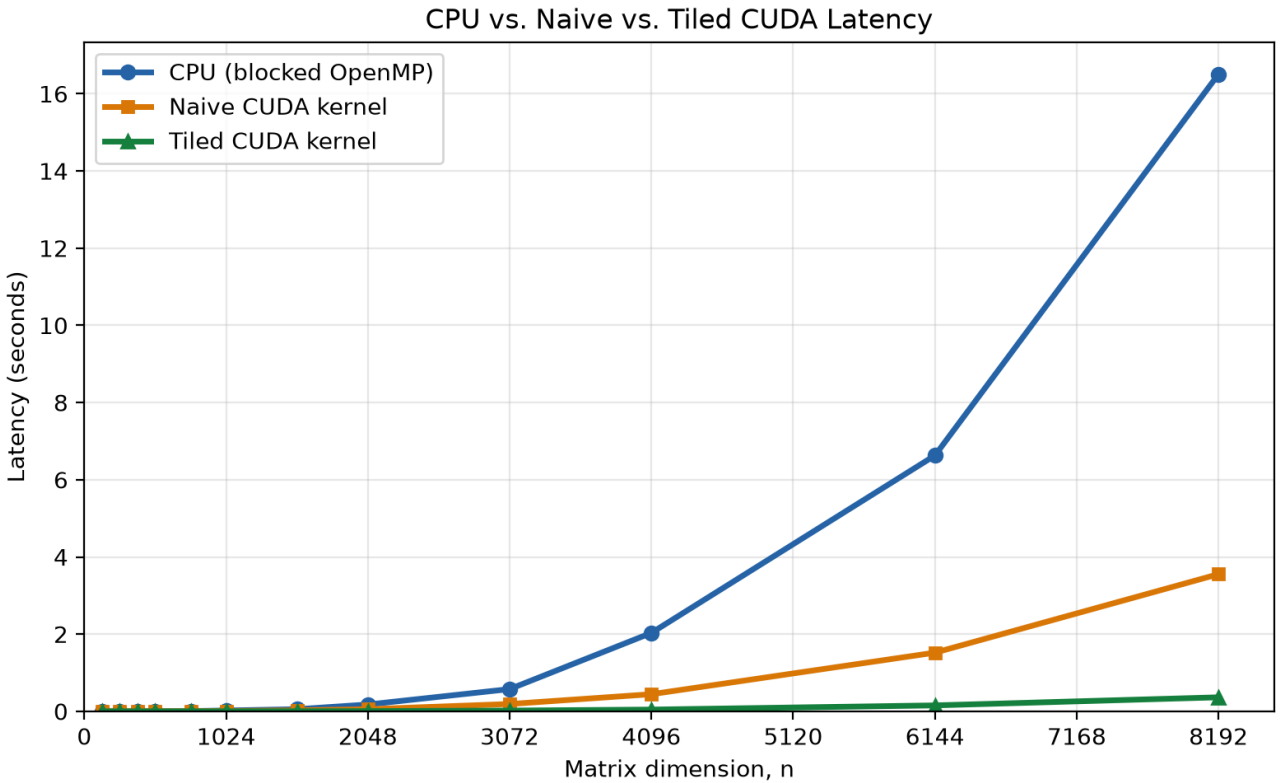
Adapted from Boehm (2022), kernel 3.

Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling cuts latency by another order of magnitude



Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B. Each tile is 32×32 elements.

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B. Each tile is 32×32 elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with 32×32 blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both A and B. Each tile is 32×32 elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM.

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}} .$$

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}} .$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.**

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}}.$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.** Even with just our **basic tiling kernel**, the observed latency of

$$\underbrace{360\text{ms}}_{\text{Observed latency}} \neq \underbrace{147\text{ms}}_{\text{Time to move data}} + \underbrace{32\text{ms}}_{\text{Theoretical minimum for work}} = 179\text{ms}.$$

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}}.$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.** Even with just our **basic tiling kernel**, the observed latency of

$$\underbrace{360\text{ms}}_{\text{Observed latency}} \neq \underbrace{147\text{ms}}_{\text{Time to move data}} + \underbrace{32\text{ms}}_{\text{Theoretical minimum for work}} = 179\text{ms}.$$

This gap comes from things like occupancy, synchronization overhead, bank conflicts...

This type of analysis is getting hard...

When we did this analysis for the **naive kernel**, it was easy to see how the data moved, and at the end of it, we even got the nice result of

$$\underbrace{3.55\text{s}}_{\text{Observed latency}} \approx \underbrace{3.52\text{s}}_{\text{Time to move data}} + \underbrace{0.031\text{s}}_{\text{Theoretical minimum for work}}.$$

However, as we apply more optimizations, **predicting performance based on hardware specifications becomes impractical.** Even with just our **basic tiling kernel**, the observed latency of

$$\underbrace{360\text{ms}}_{\text{Observed latency}} \neq \underbrace{147\text{ms}}_{\text{Time to move data}} + \underbrace{32\text{ms}}_{\text{Theoretical minimum for work}} = 179\text{ms}.$$

This gap comes from things like occupancy, synchronization overhead, bank conflicts... **Without profiling every possible source of latency, can we still determine whether memory is the bottleneck**?

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example (Car performance)

The trip took us 2 hours.

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example (Car performance)

The trip took us 2 hours.

Cal Poly Pomona $\rightarrow$ Mt. SAC

$$\frac{\sim 7 \text{ miles}}{2 \text{ hours}} \approx 3.5 \text{ mph}$$

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example (Car performance)

The trip took us 2 hours.

Cal Poly Pomona $\to$ Mt. SAC

$$\frac{\sim 7 \text{ miles}}{2 \text{ hours}} \approx 3.5 \text{ mph}$$

Cal Poly Pomona $\to$ Cal Poly SLO

$$\frac{\sim 200 \text{ miles}}{2 \text{ hours}} \approx 100 \text{ mph}$$

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example

Peak throughput of our RTX 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs. Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\text{TFLOP/s}.$$

Roughly 10% of peak throughput.

Arithmetic Intensity

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example

For $n, k, m = 8192$, our tiled kernel moved a total of 137.7 GB of data from DRAM. With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \times 10^{12} \text{ FLOP}}{137.7 \times 10^{9} \text{ bytes}} \approx \boxed{8.0 \text{ FLOP/byte}}.$$

Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth.

Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth. Thus, our ridge point is

$$\text{RP} = \frac{35.6 \times 10^{12} \text{FLOP/s}}{936 \times 10^{9} \text{bytes/s}} \approx 38.0 \text{FLOPs/byte}$$

Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

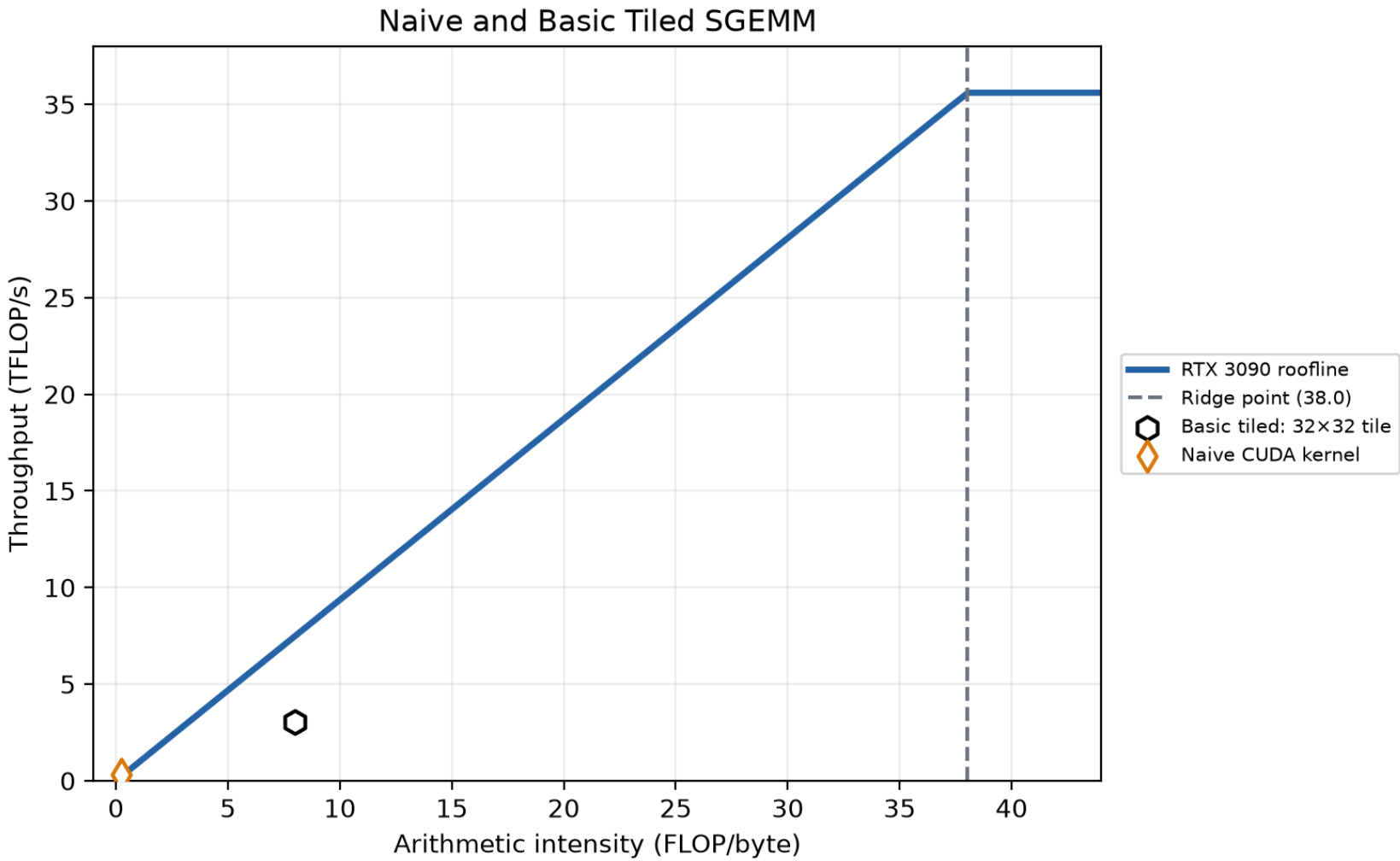
Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth. Thus, our ridge point is

$$\text{RP} = \frac{35.6 \times 10^{12} \text{FLOP/s}}{936 \times 10^{9} \text{bytes/s}} \approx 38.0 \text{FLOPs/byte}$$

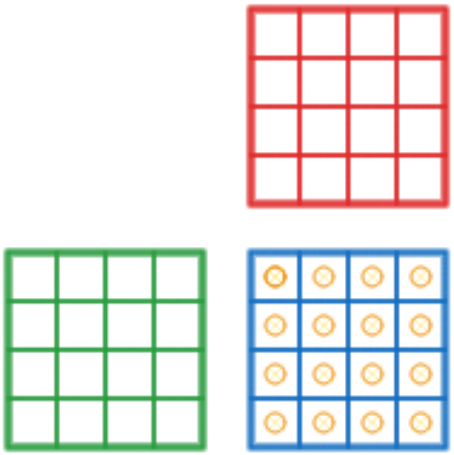
At AI $= 8.0$ FLOP/byte, we are below the ridge point, and are still memory bound.

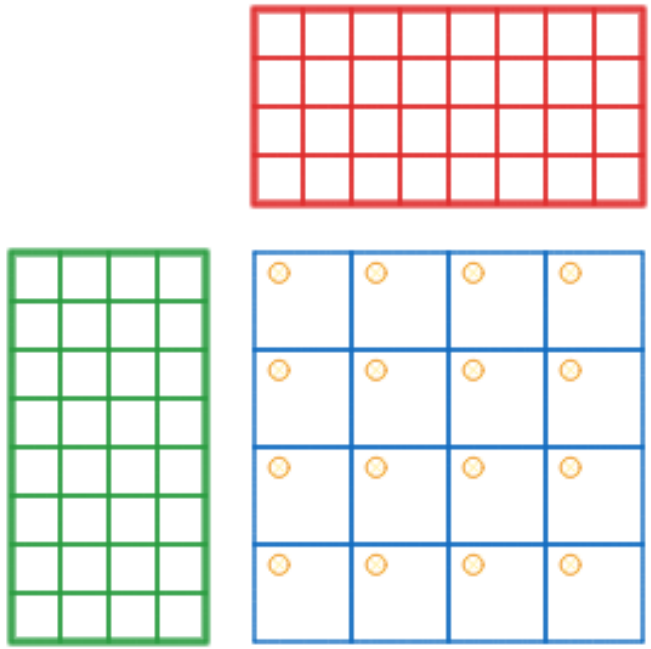
Roofline Model of our RTX 3090



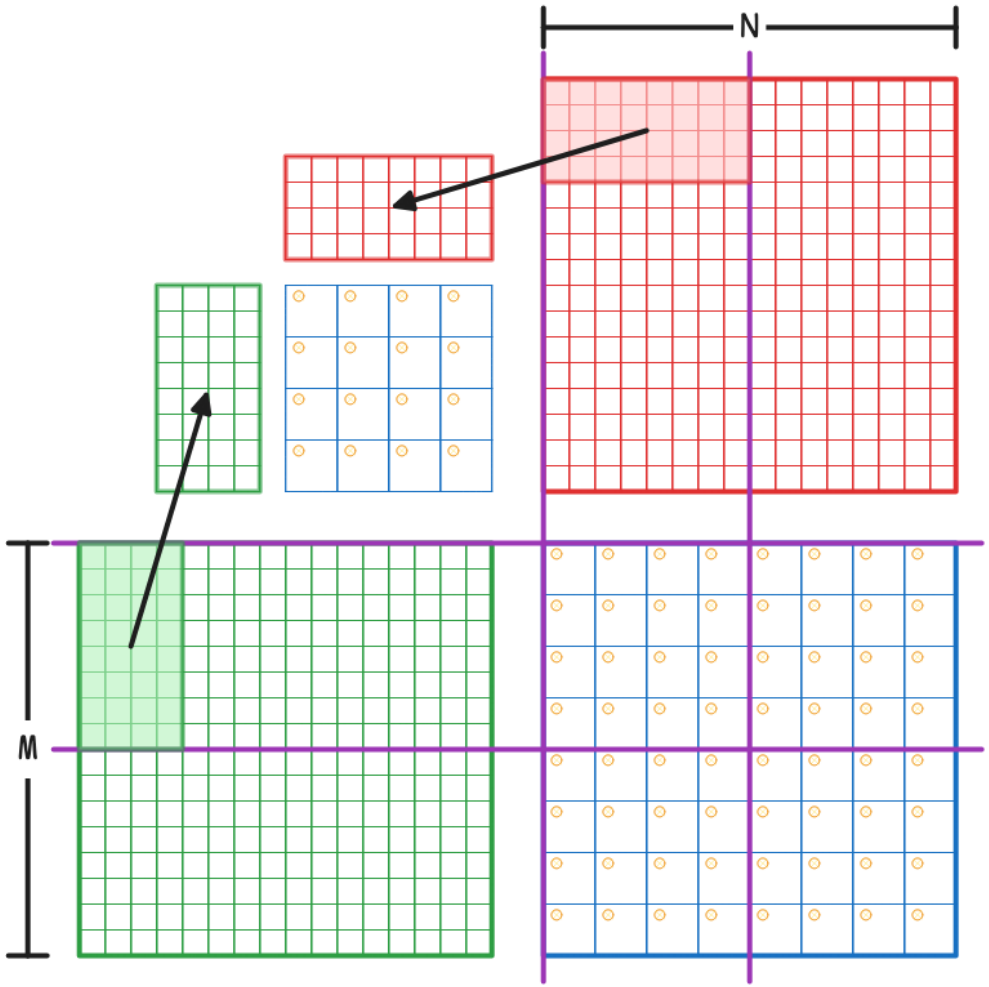
Thread coarsening: Increasing AI

- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded A value can update several columns; one loaded B value can update several rows.

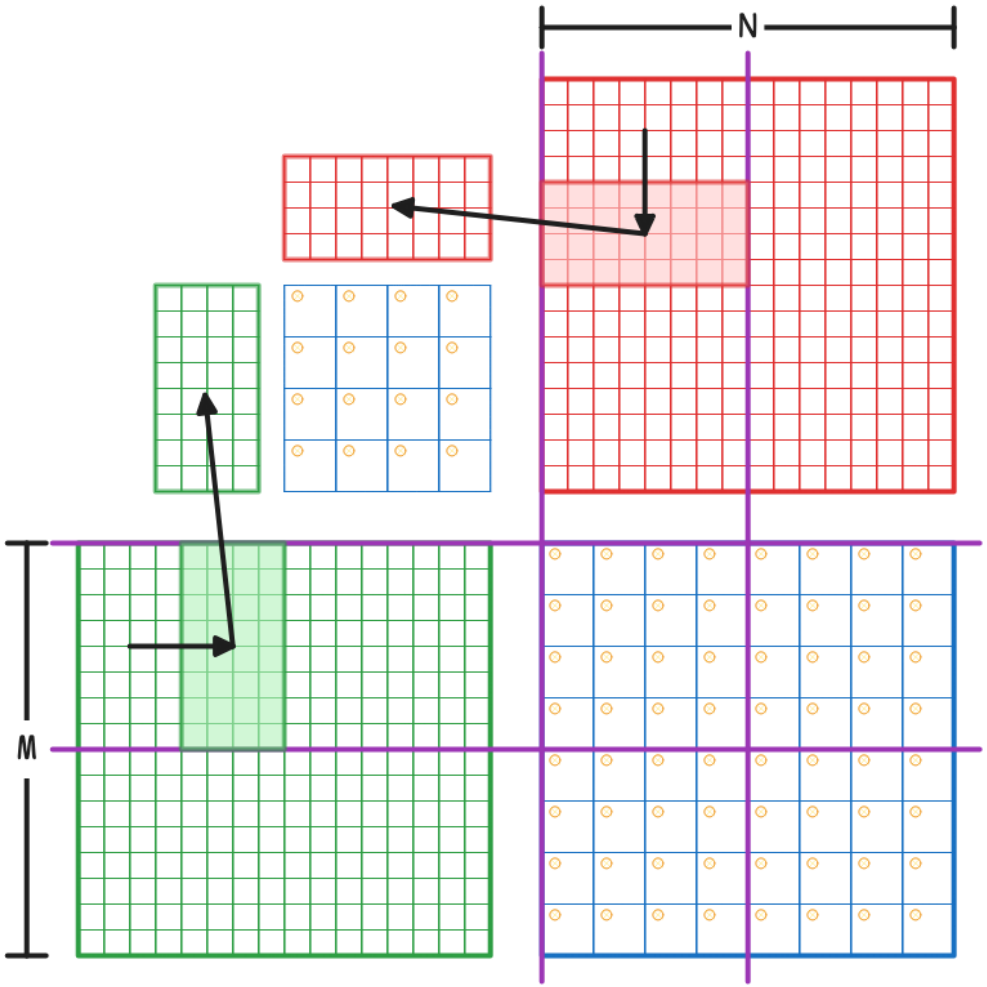




Coarsened SGEMM visualization



Coarsened SGEMM visualization



Coarsening factors are a tradeoff

Two levels of tiling

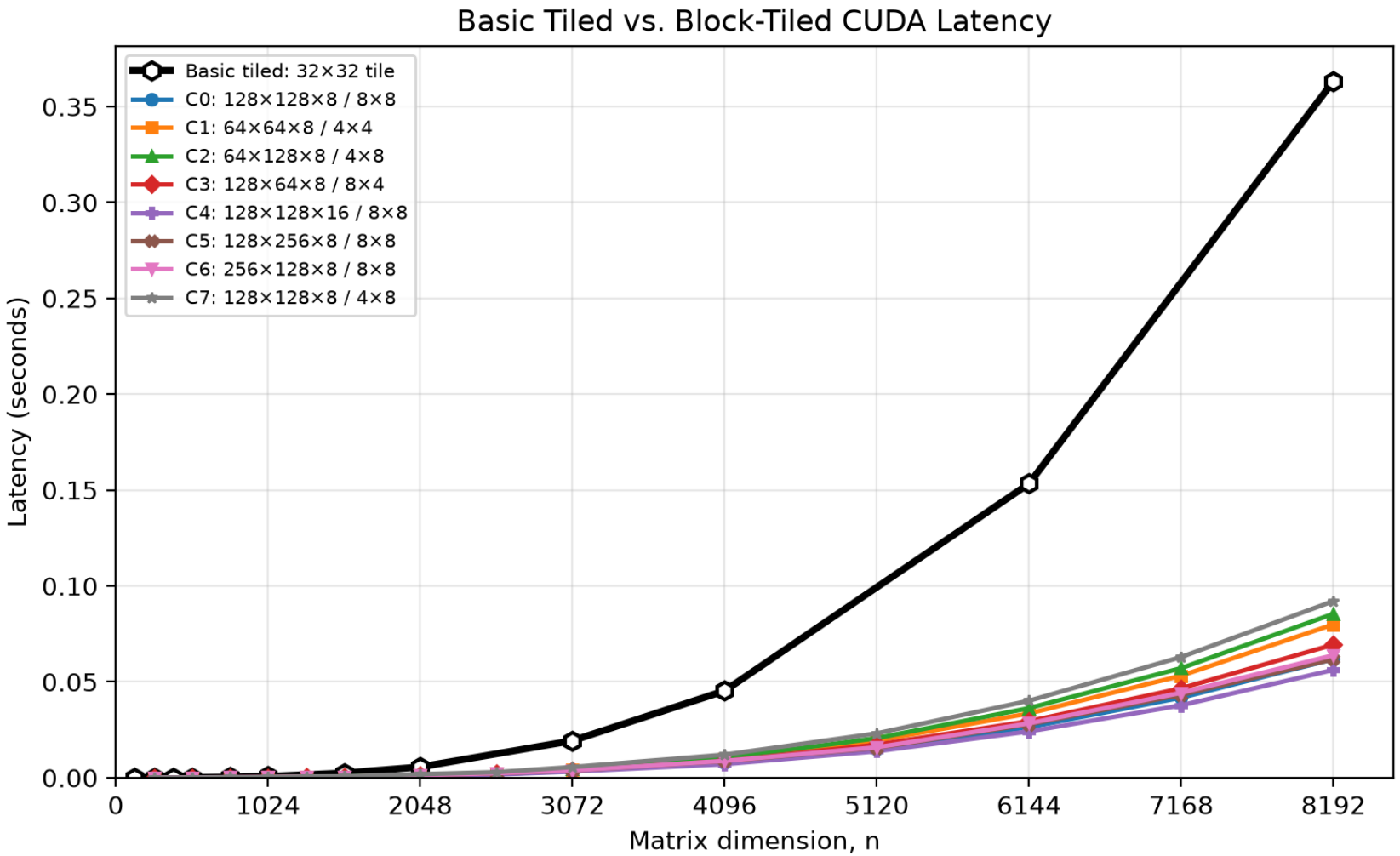
- A block computes a $BM \times BN$ output tile while advancing through K in steps of BK.
- Each thread computes a $TM \times TN$ micro-tile in registers.
- Thread count: $\frac{BM}{TM}\frac{BN}{TN}$.

Why not keep increasing them?

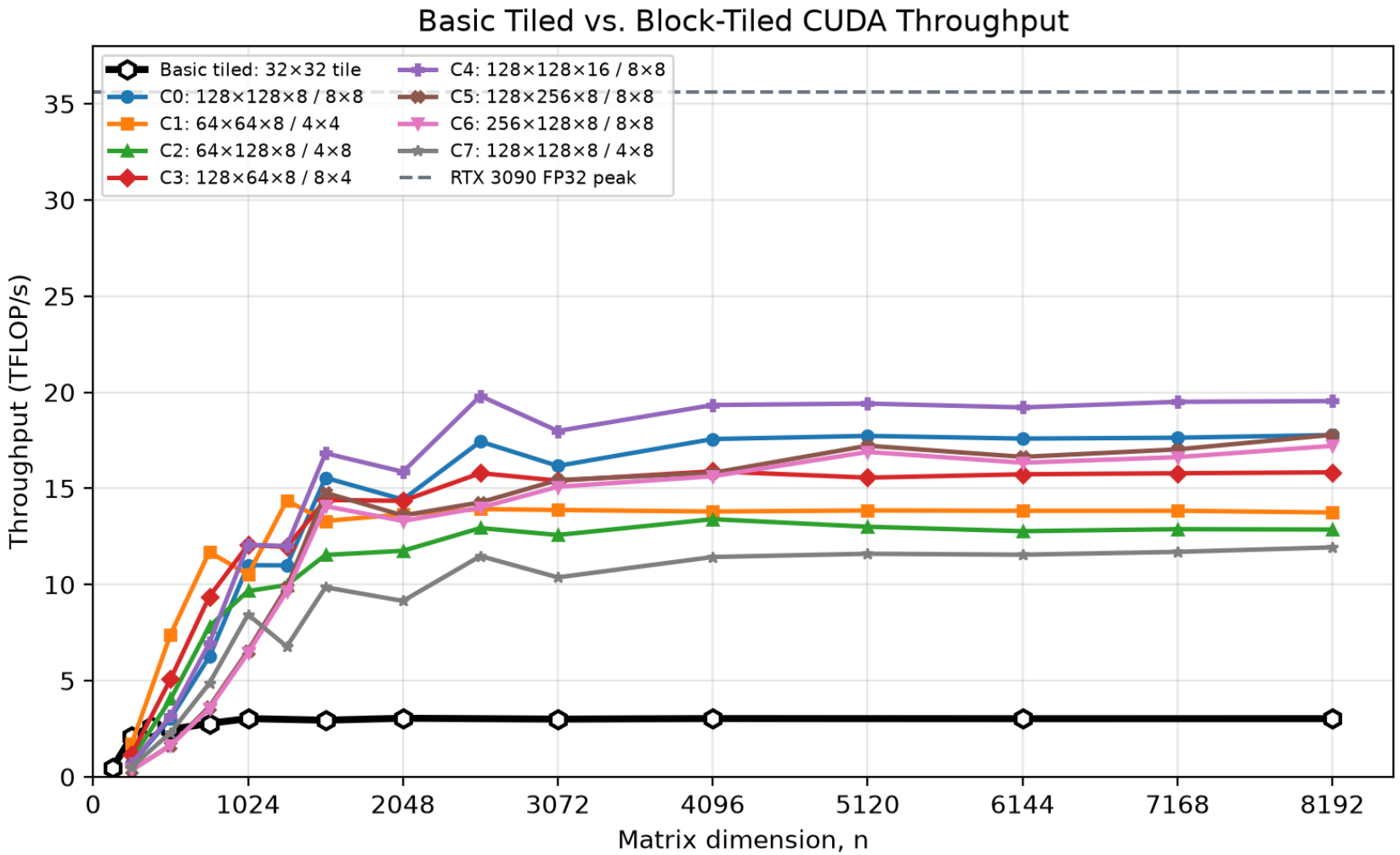
Larger tiles improve reuse and arithmetic intensity, but consume more registers and shared memory. Eventually occupancy falls and extra parallelism or reuse no longer helps.

Config.	$BM \times BN \times BK$	$TM \times TN$	AI	Throughput
C1	$64 \times 64 \times 8$	4×4	15.9	13.75 TFLOP/s
C4	$128 \times 128 \times 16$	8×8	31.5	**19.54 TFLOP/s**
C5	$128 \times 256 \times 8$	8×8	41.8	17.79 TFLOP/s

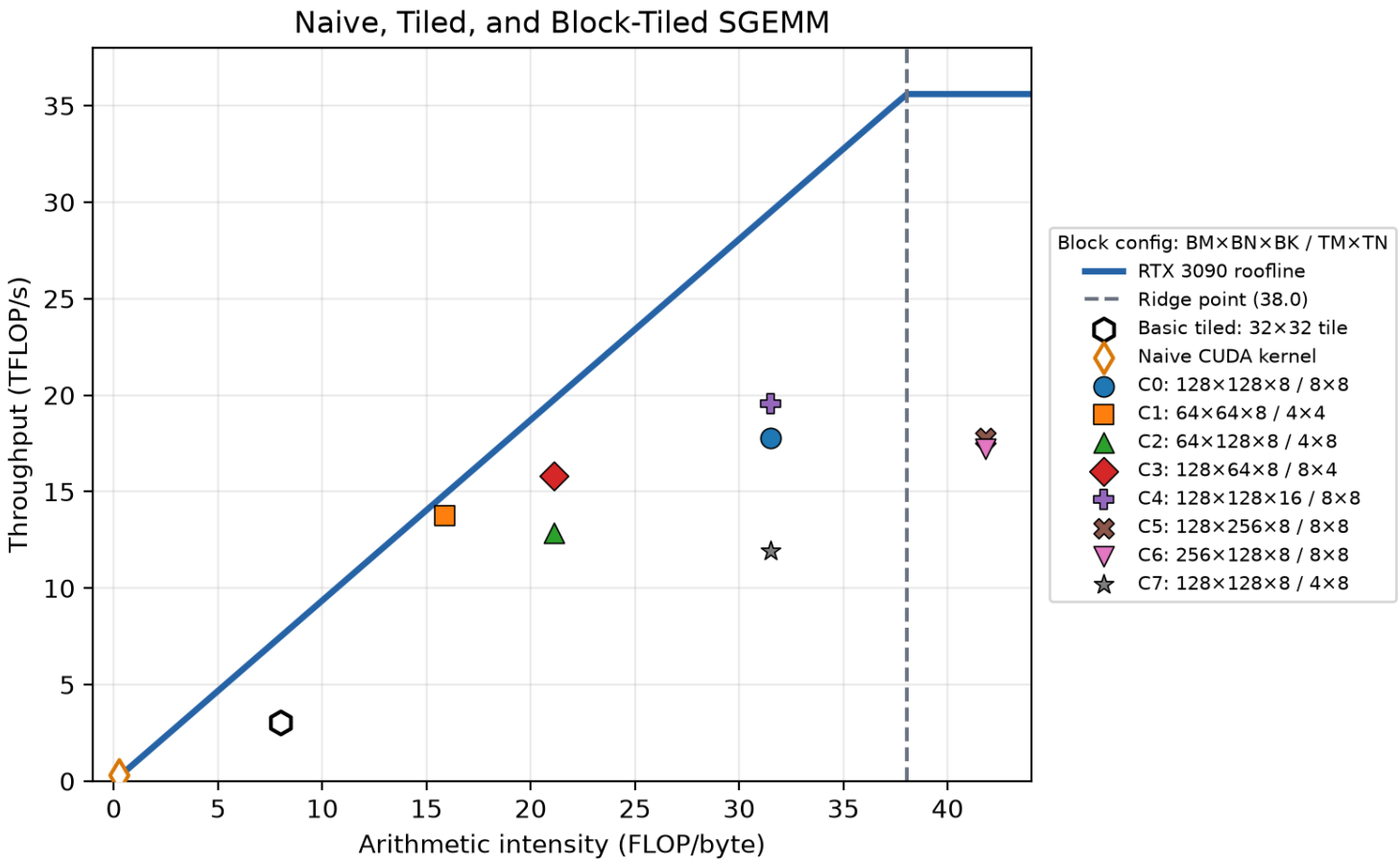
Basic tiled vs. block-tiled latency



Block tiling raises useful throughput



Tiling configurations on the roofline



Reminder that the Roofline Model only tells us the upper bound!

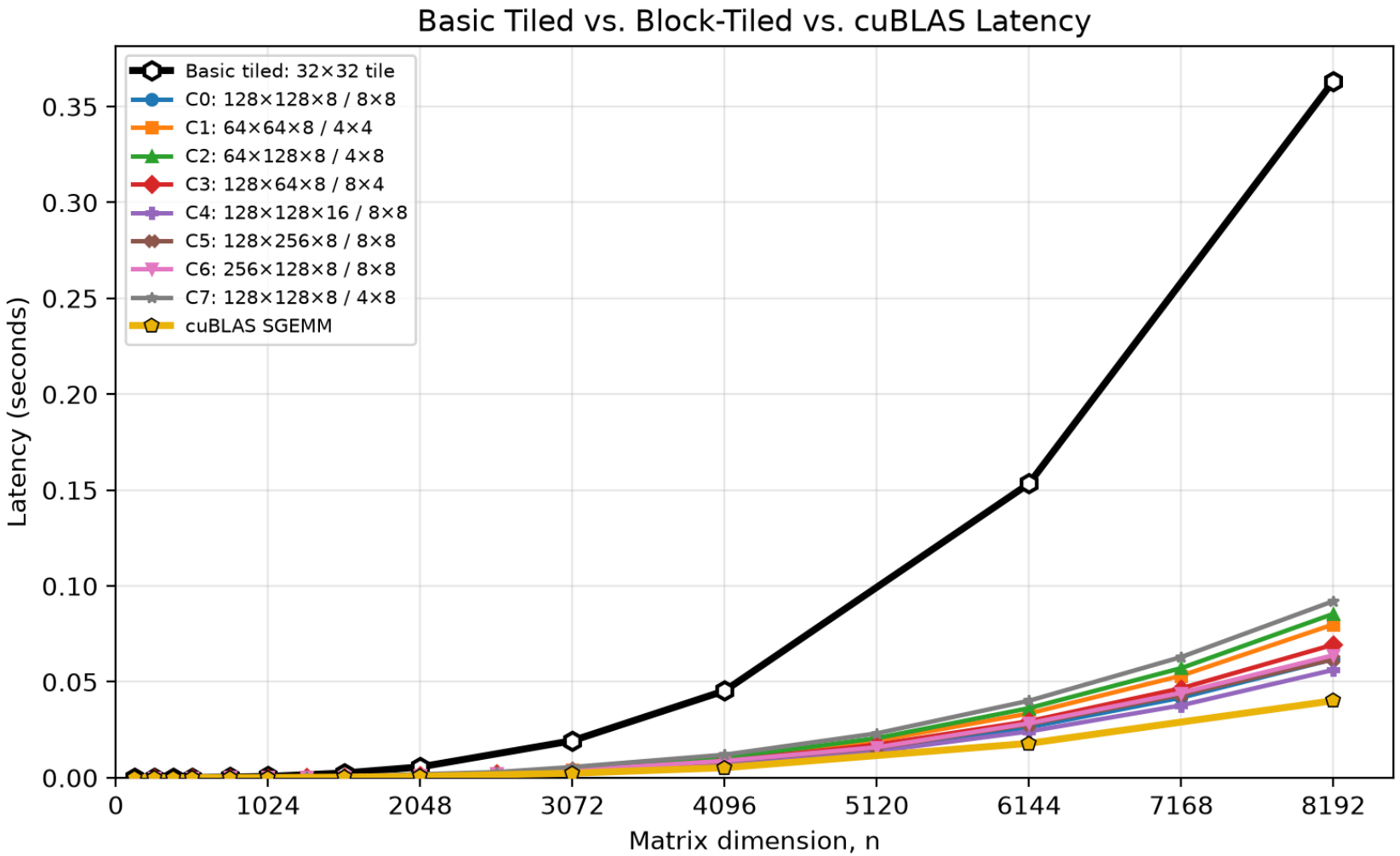
We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 27.38 TFLOP/s, while our fastest configuration (C4) reaches 19.54 TFLOP/s, or 71.4% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.2 ms versus 56.3 ms for C4, about 1.40× faster.

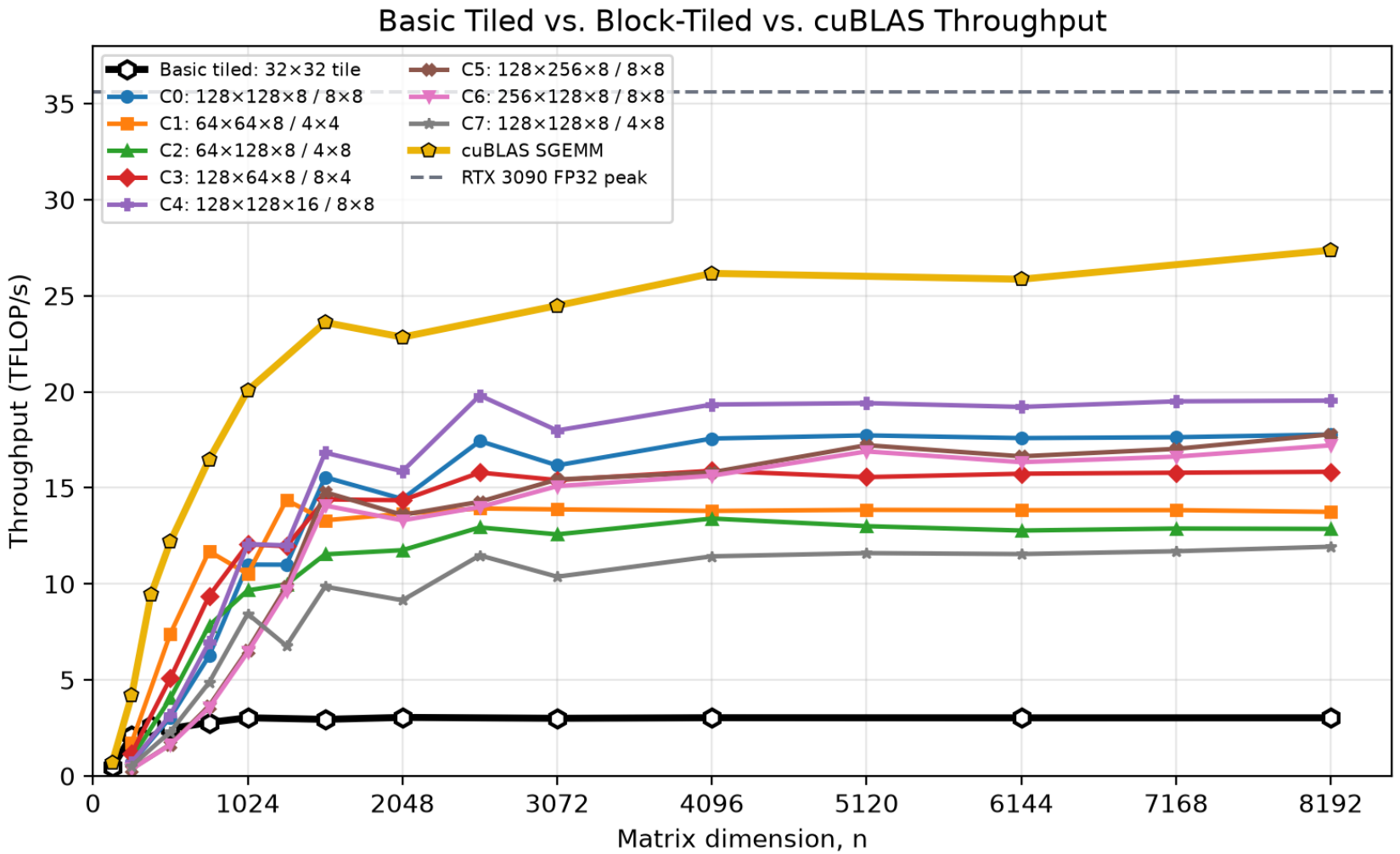
The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

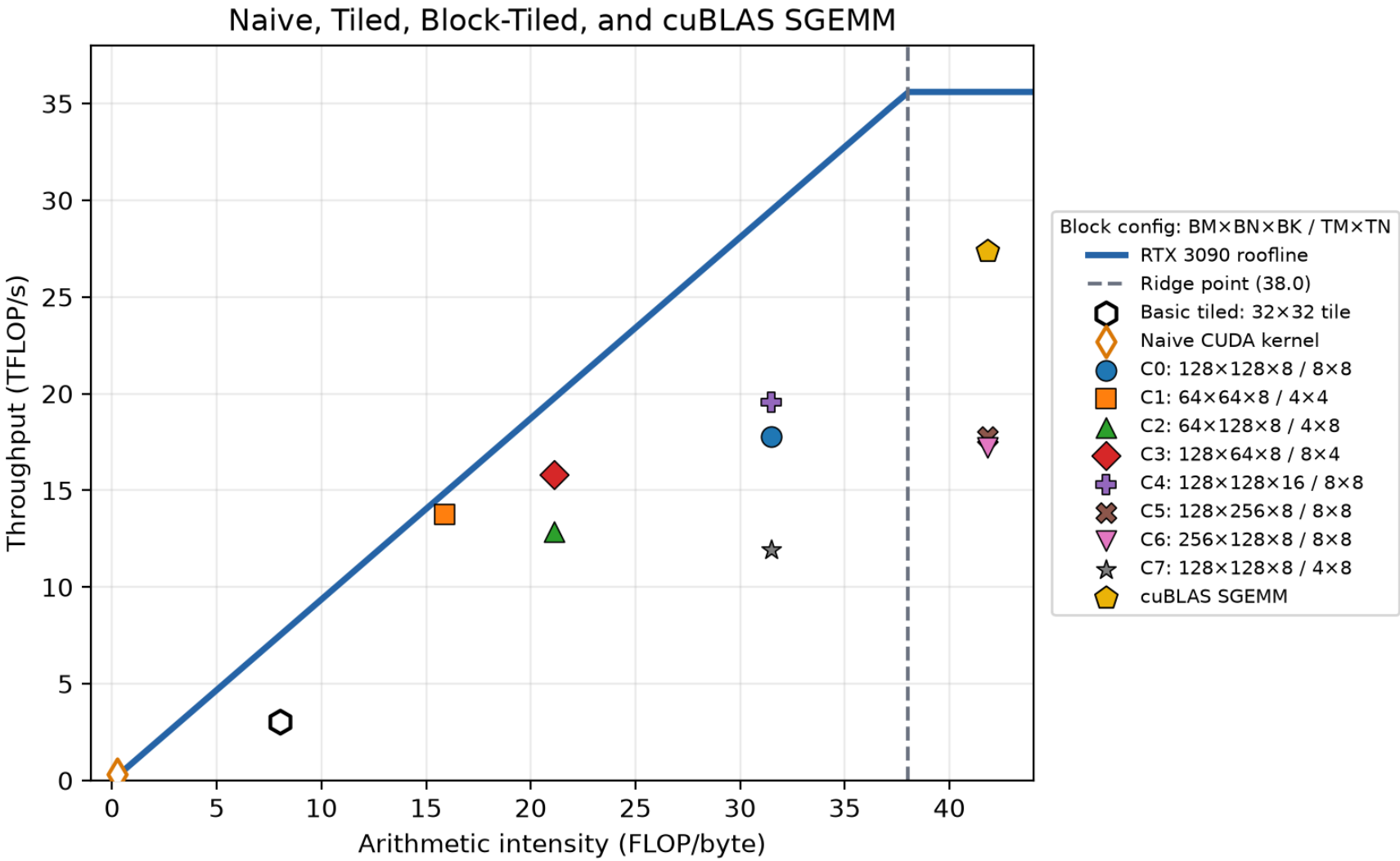
Basic tiled vs. block-tiled latency vs. cuBLAS



cuBLAS remains the performance target



cuBLAS on the roofline



cuBLAS arithmetic intensity is estimated from the profiled 256×128 output-tile shape.

We've come a long way

